LAUREA MAGISTRALE
UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA

Dipartimento di **Informatica, Sistemistica e Comunicazione**

Laurea magistrale in **informatica**

Titolo da decidere

Francesco Romeo
Matricola: 885880

Relatrice: **Dott.ssa Elisabetta Fersini**

Co-relatore: **Dottor Marco Loregian**

Anno Accademico **2025/2026**

Sommario

"abstract"

Indice

Introduzione **i**

1 Stato dell'Arte e Background Tecnologico **1**
1.1 Architetture Transformer . . . 1
1.1.1 Dalle reti neurali ricorrenti ai Transformer . . . 1
1.1.2 Il meccanismo di Self-Attention . . . 1
1.1.3 Architettura encoder-decoder e varianti decoder-only . . . 2
1.1.4 Limiti in contesto zero-shot e motivazione del RAG . . . 2
1.2 LLM: panorama e analisi comparativa . . . 2
1.2.1 Modelli proprietari . . . 2
1.2.2 Motivazioni per l'adozione di modelli locali . . . 4
1.2.3 Modelli open-weight per uso on-premise . . . 4
1.2.4 Quantizzazione e inferenza on-premise . . . 5
1.2.5 Ollama: server di inferenza locale . . . 5
1.3 Retrieval-Augmented Generation (RAG) . . . 6
1.3.1 Motivazione e definizione . . . 6
1.3.2 Architettura del sistema RAG . . . 6
1.3.3 Modelli di embedding . . . 7
1.3.4 Database vettoriali . . . 7
1.3.5 Tecniche di adattamento al dominio . . . 7
1.4 Framework di orchestrazione: Haystack . . . 9
1.5 Sintesi e posizionamento del lavoro . . . 9

2 Analisi dei Requisiti e Progettazione del Sistema **11**
2.1 Il contesto aziendale: Intré S.r.l. . . . 11
2.1.1 Il modello delle Gilde . . . 11
2.2 Caso d'uso: Proposta di nuove Gilde Intré . . . 12
2.2.1 Motivazione del caso d'uso . . . 12
2.2.2 Obiettivi del caso d'uso . . . 12
2.2.3 Descrizione dei dati e analisi del corpus . . . 13
2.3 Requisiti di sistema . . . 13
2.3.1 Vincoli architetturali . . . 14
2.4 Scelte tecnologiche: analisi comparativa . . . 15
2.4.1 Framework di orchestrazione RAG . . . 15
2.4.2 Database vettoriale . . . 16

2.4.3 Modello di embedding . . . 17
2.4.4 Server di inferenza LLM e selezione del modello di chat . . . 18
2.5 Proposta architetturale . . . 19
2.5.1 Componenti principali . . . 19
2.5.2 Progettazione della collection Qdrant . . . 21
2.5.3 Pipeline di ingestione ETL . . . 21
2.5.4 Pipeline di retrieval e logica di ranking . . . 22
2.5.5 Endpoint API di riferimento . . . 23
2.6 Integrazione con i3Guild . . . 24
2.6.1 Servizi Docker aggiuntivi . . . 24
2.6.2 Variabili d'ambiente . . . 24
2.6.3 Integrazione lato Next.js . . . 24
2.6.4 Sincronizzazione dei dati . . . 25

3 Implementazione del Sistema RAG **26**

4 Metodologia di Ricerca e Ottimizzazione **27**

5 Analisi Sperimentale e Valutazione dei Risultati **28**

Conclusioni e Sviluppi Futuri **29**

A Annotazioni e Convenzioni **30**

B Codice Utilizzato **31**

Bibliografia **32**

Ringraziamenti **32**

Introduzione

- **Contesto:** Diffusione dei Large Language Models (LLM) e le sfide dell'adozione in ambito corporate (privacy, allucinazioni, aggiornamento dei dati). → giustificare le affermazioni con citazioni e dati.
- **Il Problema:** Gestione della conoscenza non strutturata e valorizzazione del materiale formativo (Gilde) in Intré. → descrizione dell'azienda e dello use-case.
- **Obiettivo della tesi:** Progettazione e sviluppo di un sistema RAG (Retrieval-Augmented Generation) con focus sull'ottimizzazione dell'inferenza e dell'accuratezza nel dominio specifico.
- **Estrema sintesi dei risultati ottenuti**
- **NOTA:** essere esplicativi sotto tutti i punti di vista, deve essere una versione sintetica della testi - un riassunto - con l'aggiunta delle motivazioni del progetto.

Capitolo 1

Stato dell'Arte e Background Tecnologico

Il presente capitolo fornisce le basi teoriche e tecnologiche necessarie alla comprensione del sistema descritto nei capitoli successivi. Si esaminano le architetture Transformer, alla base dei modelli linguistici (*Large Language Models*, LLM) considerati nel progetto, confrontando modelli open source e proprietari e valutando le soluzioni idonee all'esecuzione on-premise. Si presenta inoltre il paradigma *Retrieval-Augmented Generation* (RAG), ne viene descritta l'architettura e sono illustrate le principali tecniche di adattamento al dominio per mitigare i limiti intrinseci degli LLM.

1.1 Architetture Transformer

1.1.1 Dalle reti neurali ricorrenti ai Transformer

Il modello Transformer, introdotto da Vaswani et al. nel 2017 [**vaswani2017attention**], ha ridefinito lo stato dell'arte nel trattamento del linguaggio naturale (*Natural Language Processing*, NLP). I sistemi precedenti si basavano su reti neurali ricorrenti (*Recurrent Neural Networks*, RNN) e, più specificamente, su architetture LSTM (*Long Short-Term Memory*) [**hochreiter1997lstm**], che elaboravano i dati in modo sequenziale, accumulando uno stato nascosto ad ogni passo temporale. Questa formulazione presentava due limitazioni principali: l'impossibilità di parallelizzare il calcolo durante l'addestramento e la difficoltà di mantenere dipendenze a lungo termine in sequenze molto lunghe (*vanishing gradient*).

Il Transformer affronta entrambe le limitazioni abbandonando la ricorrenza a favore di un meccanismo di *self-attention* globale, che consente a ciascun elemento di una sequenza di relazionarsi direttamente con tutti gli altri, indipendentemente dalla distanza posizionale.

1.1.2 Il meccanismo di Self-Attention

Il nucleo computazionale del Transformer è il meccanismo di *attention* scalato. Data una sequenza di token rappresentati come vettori di dimensione d_{model}, si calcolano tre proiezioni lineari per ogni elemento: la query $\mathbf{Q}$, la chiave $\mathbf{K}$ e il valore $\mathbf{V}$. Il punteggio di attenzione è definito come [**vaswani2017attention**]:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{1.1}$$

dove d_k è la dimensione dei vettori di chiave; il fattore $1/\sqrt{d_k}$ previene la saturazione della funzione softmax in presenza di prodotti scalari molto grandi.

Il meccanismo viene replicato su più *teste* in parallelo, ciascuna delle quali apprende una proiezione diversa dello stesso spazio di input (*Multi-Head Attention*). La concatenazione degli output di h teste permette al modello di catturare simultaneamente relazioni sintattiche, semantiche e posizionali:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\,\mathbf{W}^O \tag{1.2}$$

con $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$.

La posizione dei token nella sequenza non è catturata dall'attention; viene iniettata tramite *positional encoding*, ossia vettori sommati agli embedding di input che forniscono informazioni sull'ordine relativo degli elementi.

1.1.3 Architettura encoder-decoder e varianti decoder-only

L'architettura originale del Transformer è di tipo encoder-decoder: un encoder trasforma la sequenza di input in una rappresentazione contestuale densa, mentre un decoder la utilizza — attraverso un meccanismo di *cross-attention* — per generare la sequenza di output. Questa struttura è particolarmente adatta a compiti *sequence-to-sequence*, come la traduzione automatica [**vaswani2017attention**].

I modelli linguistici generativi contemporanei adottano prevalentemente un' architettura *decoder-only*, nella quale l'attenzione è applicata in modo *causale* (o *masked*): ogni token dipende esclusivamente dai precedenti. Questo vincolo è necessario per l'addestramento autoregressivo, nel quale il modello impara a predire il token successivo dato il contesto.

1.1.4 Limiti in contesto zero-shot e motivazione del RAG

Nonostante l'efficacia dei modelli Transformer pre-addestrati, la loro applicazione in contesti aziendali specifici presenta limiti pratici, sintetizzati nella Tabella 1.1.

Questi fattori motivano l'adozione del paradigma RAG, discusso nella Sezione 1.3, come alternativa pratica al fine-tuning per l'adattamento al dominio.

1.2 LLM: panorama e analisi comparativa

L'ecosistema degli LLM si è diversificato rapidamente a partire dal 2022, con una distinzione netta tra modelli proprietari, accessibili tramite API cloud, e modelli open-weight i cui pesi sono scaricabili e utilizzabili on-premise — spesso con vincoli di licenza per l'uso commerciale. La Tabella 1.2 fornisce una panoramica dei principali modelli considerati.

1.2.1 Modelli proprietari

I modelli proprietari offrono prestazioni elevate sui benchmark generali, ma presentano vincoli rilevanti per l'adozione in contesti con requisiti di riservatezza.

Tabella 1.1: Limiti strutturali degli LLM in contesti aziendali e relative strategie di mitigazione.

Limitazione	**Descrizione**	**Strategia di mitigazione**
Conoscenza statica	I parametri codificano la conoscenza fino al *knowledge cutoff*; nessun accesso a dati interni o aggiornati dopo l'addestramento.	RAG: recupero dinamico da knowledge base esterna, aggiornabile senza riaddestrare il modello.
Allucinazioni [**ji2023survey**]	Il modello può generare testo plausibile ma fattualmente errato su informazioni assenti dalla distribuzione di addestramento.	RAG con *grounding*: la risposta è ancorata a documenti recuperati e verificabili.
Privacy dei dati	Il fine-tuning su modelli cloud può esporre dati riservati a infrastrutture di terze parti, con rischi di conformità normativa.	Esecuzione on-premise con modelli open-weight: nessuna trasmissione di dati verso servizi esterni.

Tabella 1.2: Panorama comparativo dei principali LLM (2024–2025). † Esecuzione garantita on-premise senza dipendenze cloud; ‡ pesi scaricabili pubblicamente; MoE = *Mixture of Experts*.

Modello	**Sviluppatore**	**Licenza**	**Accesso**	**Architettura / Param. attivi**	**Ctx**	**Multi-modale**
GPT-5 / o3	OpenAI	Proprietaria	API cloud	Dense, n.d.	128k	✓
Claude 4 Sonnet	Anthropic	Proprietaria	API cloud	Dense, n.d.	200k	✓
Gemini 3 Pro	Google DM	Proprietaria	API cloud	Dense, n.d.	1M	✓
Llama 4 Scout	Meta AI	Open-weight‡	Self-hosted†	MoE, 17B	10M	✓
Mistral Large 3	Mistral AI	Apache 2.0‡	Self-hosted†	MoE, ~40B	128k	✓
Qwen3-8B	Alibaba Cloud	Apache 2.0‡	Self-hosted†	Dense, 8B	128k	–

GPT-5 e OpenAI o-series. La famiglia GPT-5 [**openai2023gpt4**] rappresenta un riferimento per la pianificazione complessa e l'integrazione multimodale. La serie "o" (`o1`, `o3`) integra tecniche di ragionamento come il *chain-of-thought* durante l'inferenza, migliorando le prestazioni in compiti scientifici e logico-matematici [**openai2024o1**]. L'erogazione è vincolata ad API cloud proprietarie; in ambito enterprise è possibile operare tramite servizi dedicati (es. Azure OpenAI Service) che offrono garanzie contrattuali di isolamento dei dati.

Claude 4 (Anthropic). I modelli Claude adottano approcci come la *Constitutional AI* [**bai2022constitutional**], un protocollo di addestramento guidato da principi per migliorare l'allineamento. Anche questi modelli sono erogati in modalità cloud; Anthropic propone modalità contrattuali (ad esempio *Zero Data Retention*) per rispondere a esigenze enterprise [**anthropic2025claude4**].

Gemini (Google DeepMind). La famiglia Gemini [**google2025gemini25**] è caratterizzata da multimodalità nativa e da finestre di contesto molto estese (fino a 1M di token per Gemini 3 Pro). Anche in questo caso l'accesso è fornito tramite servizi cloud.

1.2.2 Motivazioni per l'adozione di modelli locali

Nonostante le prestazioni dei modelli proprietari, il sistema descritto in questa tesi utilizza modelli eseguiti interamente in locale (*self-hosted*). La scelta si basa su tre motivazioni principali, sintetizzate nella Tabella 1.3.

Tabella 1.3: Motivazioni per l'adozione di modelli LLM on-premise.

Motivazione	**Descrizione**	**Riferimento normativo / letteratura**
Sovranità del dato e conformità	L'esecuzione locale evita la trasmissione di dati verso infrastrutture di terze parti, anche in presenza di garanzie contrattuali (ZDR, DPA).	GDPR [**regulation2016gdpr**]; rischi del cloud nella gestione di dati sensibili.
Costi operativi nel lungo periodo	Il pricing a consumo dei provider cloud può risultare più oneroso rispetto all'ammortamento dell'hardware locale in scenari ad alto volume.	[**bommasani2021opportunities**]; rischio di *vendor lock-in*.
Controllo e personalizzazione	I modelli open-weight (LLaMA, Mistral, Qwen) consentono fine-tuning supervisionato e allineamento su dati propri, senza le restrizioni delle policy dei sistemi proprietari.	[**touvron2023llama**, **jiang2023mistral**, **qwen2024**]

Per queste ragioni, la trattazione si concentra successivamente sull'analisi di modelli eseguibili localmente, riservando ai sistemi proprietari il ruolo di riferimento comparativo per la valutazione delle prestazioni nel Capitolo **??**.

1.2.3 Modelli open-weight per uso on-premise

Llama 4 (Meta AI). Rilasciata nell'aprile 2025, la famiglia Llama 4 [**meta2025llama4**] adotta architetture *Mixture of Experts* (MoE) e supporta la multimodalità. Le varianti *Scout* e *Maverick* bilanciano parametri effettivi e numero di esperti per gestire contesti estesi fino a 10 milioni di token. La licenza open-weight presenta clausole specifiche da valutare in fase di adozione, in particolare per operatori e per l'Unione Europea.

Mistral 3 (Mistral AI). Nel 2025 Mistral AI ha ampliato la propria offerta con modelli orientati all'integrazione locale [**mistral2025large3**]. Il modello di punta, *Mistral Large 3*, è un modello MoE multilingue; la famiglia include varianti ottimizzate per l'esecuzione su hardware locale. La distribuzione sotto licenza Apache 2.0 facilita l'impiego in progetti aperti.

Qwen3 (Alibaba Cloud). La generazione Qwen3 [**qwen2025qwen3**] introduce modalità operative differenziate per task logici ed efficienza. La famiglia copre modelli da dimensioni molto ridotte fino a varianti MoE con finestre di contesto estese. Nel Capitolo **??** si descrive la scelta della variante 8B per il sistema proposto, motivata dal bilancio tra qualità semantica e velocità di inferenza su CPU-only.

1.2.4 Quantizzazione e inferenza on-premise

Un elemento centrale per l'esecuzione locale di LLM è la quantizzazione: la riduzione della precisione numerica dei pesi del modello da FP32 o FP16/BF16 a rappresentazioni a bassa precisione (8-bit, 4-bit). La Tabella 1.4 riassume l'impatto delle principali tecniche su memoria e qualità per un modello da 7B parametri.

Tabella 1.4: Impatto della quantizzazione su occupazione di memoria e qualità per un modello da 7B parametri. I valori sono indicativi e tratti da [**dettmers2023qlora**, **frantar2022gptq**].

Formato	**Bit/peso**	**VRAM (7B)**	**Degr. qualità**	**Tecnica**
FP32	32	~28 GB	baseline (0%)	Standard
FP16/BF16	16	~14 GB	≈ 0%	Standard
GPTQ	8	~8 GB	<1%	[**frantar2022gptq**]
GGUF Q4_K_M	4	~4 GB	1–3%	llama.cpp
NF4 (QLoRA)	4	~4 GB	<1%	[**dettmers2023qlora**]

L'impatto della quantizzazione sulla qualità della generazione in contesti specifici è analizzato nel Capitolo **??**.

> Il 24 marzo 2026 Google Research ha presentato TurboQuant [**google2026turboquant**], un framework di compressione data-oblivious progettato per ridurre i colli di bottiglia mnemonici della KV Cache[1] nei Large Language Models (LLM). La tecnologia mostra la capacità di ridurre il footprint della memoria di un fattore rilevante (ordine di grandezza riportato dagli autori) e di migliorare l'efficienza computazionale su acceleratori recenti, senza degradazione apprezzabile della precisione nelle valutazioni pubblicate.
>
> Il metodo si struttura in due stadi principali: PolarQuant, che normalizza la distribuzione dei dati mediante una proiezione in coordinate polari, e il Quantized Johnson-Lindenstrauss (QJL), un correttore che applica dimensionamento proiettivo per limitare l'errore introdotto dalla quantizzazione. PolarQuant evita di memorizzare per ogni blocco scale e zero-point grazie a una rotazione seguita da una quantizzazione vettoriale; QJL fornisce una correzione a bassa complessità per preservare le distanze rilevanti nei prodotti scalari. Data la natura training-free e l'efficienza nella gestione dei tensori di attenzione, l'integrazione di TurboQuant può facilitare l'estensione delle finestre di contesto in locale, rendendolo un elemento rilevante per il progetto presentato in questo lavoro.

1.2.5 Ollama: server di inferenza locale

Per il progetto in esame, l'infrastruttura di inferenza è gestita tramite **Ollama** [**ollama2023**], un server open source che espone modelli locali in formato GGUF tramite un'API REST compatibile con la specifica *OpenAI Chat Completions*. Ollama semplifica il ciclo di vita dei modelli locali: l'installazione avviene tramite un comando semplice (`ollama pull <modello>`), la gestione del

[1] KV Cache: struttura dati che memorizza i tensori Key e Value durante l'inferenza autoregressiva degli LLM. Poiché la sua dimensione cresce linearmente con la lunghezza del contesto ($O(n)$), essa rappresenta il principale limite fisico per il deployment di modelli a lungo contesto su hardware con VRAM limitata.

contesto e del formato di prompt è automatizzata, e il server può essere distribuito come container Docker. Nel contesto di questo progetto, Ollama serve sia il modello di chat (`qwen3:8b`) sia il modello di embedding (`nomic-embed-text`), il cui ruolo è descritto nella Sezione 1.3.3.

1.3 Retrieval-Augmented Generation (RAG)

1.3.1 Motivazione e definizione

Il paradigma *Retrieval-Augmented Generation* (RAG), formalizzato da Lewis et al. [**lewis2020retrieval**], nasce per superare il limite dei LLM come sistemi a conoscenza chiusa (*closed-book*): la conoscenza del modello è codificata nei parametri e non può essere aggiornata senza un nuovo ciclo di addestramento o fine-tuning.

In un sistema RAG, la generazione è affiancata da una fase di *retrieval* dinamico: prima di invocare l'LLM si esegue una ricerca in una base di conoscenza esterna per recuperare documenti pertinenti alla query. I documenti recuperati vengono inseriti nel contesto del prompt (*prompt augmentation*), in modo che la risposta del modello si basi anche su informazioni aggiornate e verificabili, riducendo il rischio di allucinazioni [**shuster2021retrieval**].

1.3.2 Architettura del sistema RAG

Un sistema RAG si compone di due pipeline distinte, messe a confronto nella Tabella 1.5 e illustrate nella Figura 1.1: la pipeline di **ingestione** (offline) e la pipeline di **inferenza** (real-time).

Tabella 1.5: Le due pipeline del sistema RAG a confronto.

Fase	**Ingestion pipeline (offline)**	**Query pipeline (real-time)**
1	Pre-processing e chunking dei documenti sorgente	Embedding della query utente nello stesso spazio vettoriale dei documenti
2	Generazione degli embedding tramite modello locale (`nomic-embed-text`)	Ricerca ANN nel database vettoriale (*top-k*) con filtri su metadati
3	Indicizzazione in Qdrant con payload di metadati strutturati	Augmented generation: contesto recuperato + query → LLM
Trigger	Esecuzione manuale o schedulata (es. giornaliera)	Ad ogni richiesta utente
Latenza	Minuti / ore (elaborazione batch)	Millisecondi (inferenza real-time)

Pipeline di ingestione

La pipeline di ingestione trasforma i documenti grezzi in vettori persistiti nel database vettoriale. Le fasi principali sono le seguenti.

Pre-processing e chunking. Il documento viene suddiviso in unità testuali (*chunk*) di dimensione controllata. La strategia di chunking influisce direttamente sulla qualità del retrieval: chunk troppo piccoli perdono contesto, mentre chunk troppo grandi possono diluire il segnale semantico e superare la finestra di contesto del modello di embedding [**chen2023benchmarking**].

Generazione degli embedding. Ogni chunk viene trasformato in un vettore denso da un modello di embedding $f: \mathcal{T} \to \mathbb{R}^d$, in modo che documenti semanticamente simili producano vettori vicini secondo la similarità coseno:

$$\mathrm{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|} \tag{1.3}$$

Indicizzazione nel database vettoriale. I vettori vengono persistiti in **Qdrant** [**qdrant2023**], che implementa l'algoritmo HNSW (*Hierarchical Navigable Small World*) [**malkov2018efficient**] per la ricerca approssimata (*Approximate Nearest Neighbor*, ANN). La scelta di Qdrant rispetto ad alternative quali pgvector, Chroma e Milvus è motivata nel Capitolo **??**.

Pipeline di inferenza

Embedding della query. La query dell'utente viene proiettata nello stesso spazio vettoriale dei documenti indicizzati con lo stesso modello di embedding.

Retrieval. Si esegue una ricerca ANN in Qdrant per recuperare i k documenti più simili alla query. La ricerca può essere limitata tramite *metadata filtering* a sottoinsiemi coerenti (ad esempio, documenti appartenenti a una specifica epoca temporale).

Augmented generation. Il contesto recuperato viene concatenato alla query nel prompt inviato all'LLM. Il modello genera la risposta basandosi sulla propria conoscenza parametrica e sulle informazioni esterne fornite.

1.3.3 Modelli di embedding

La scelta del modello di embedding influisce sulla qualità del retrieval. Il vincolo di esecuzione on-premise restringe la scelta a modelli distribuibili localmente tramite Ollama. La valutazione comparativa è stata condotta su quattro candidati — `nomic-embed-text`, `mxbai-embed-large`, `all-minilm` e `multilingual-e5-large` — secondo criteri di qualità per la lingua italiana, occupazione di memoria, compatibilità con Ollama e punteggi sul benchmark MTEB (*Massive Text Embedding Benchmark*) [**muennighoff2022mteb**]. L'analisi completa e la motivazione della scelta finale di `nomic-embed-text` [**nussbaum2024nomic**] sono presentate nel Capitolo **??**.

1.3.4 Database vettoriali

Un database vettoriale è un sistema di storage specializzato per vettori densi ad alta dimensionalità, progettato per rispondere a query di tipo *k-nearest neighbor* (kNN). A differenza dei database relazionali, dove le query sono esatte, la ricerca vettoriale è per natura approssimata: si cercano i vettori più vicini accettando un trade-off tra richiamo (*recall*) e velocità (*throughput*).

L'algoritmo HNSW costruisce un grafo gerarchico multi-livello nello spazio vettoriale, con complessità di ricerca approssimativamente $O(\log n)$ rispetto al numero di punti n — contro $O(n)$ della ricerca lineare esatta [**malkov2018efficient**] — garantendo scalabilità anche per corpora ampi.

1.3.5 Tecniche di adattamento al dominio

L'adattamento di un LLM pre-addestrato a un dominio specifico può avvenire tramite approccio parametrico (*fine-tuning*) o non parametrico (RAG, *prompt engineering*). Questa sezione analizza

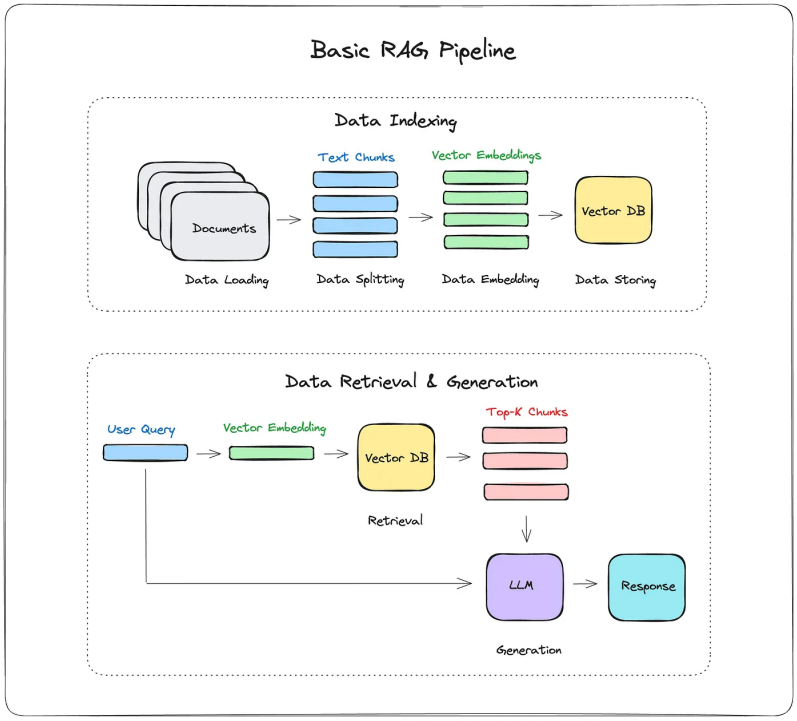

Figura 1.1: Schema della pipeline RAG: la sezione superiore rappresenta la fase di ingestione offline; la sezione inferiore la pipeline di inferenza in tempo reale [**lewis2020retrieval**].

le tecniche di fine-tuning a parametri ridotti, che costituiscono il riferimento teorico per il confronto discusso nel Capitolo **??**.

Fine-tuning con adattatori: LoRA

Il fine-tuning completo di un LLM richiede risorse computazionali elevate. Hu et al. [**hu2022lora**] hanno proposto *Low-Rank Adaptation* (LoRA), che vincola gli aggiornamenti dei pesi a matrici di rango basso. Dato un layer lineare con matrice $\mathbf{W}_0 \in \mathbb{R}^{m \times n}$, LoRA introduce:

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{B}\mathbf{A} \tag{1.4}$$

con $\mathbf{A} \in \mathbb{R}^{r \times n}$, $\mathbf{B} \in \mathbb{R}^{m \times r}$ e $r \ll \min(m, n)$. I pesi originali $\mathbf{W}_0$ rimangono congelati; solo $\mathbf{A}$ e $\mathbf{B}$ vengono ottimizzati. Con $r = 8$ e $m = n = 4096$, il numero di parametri addestrabili si riduce di circa due ordini di grandezza rispetto al fine-tuning completo.

QLoRA: fine-tuning a 4-bit

Quantized LoRA (QLoRA) [**dettmers2023qlora**] combina la quantizzazione a 4-bit del modello base con adattatori LoRA a precisione piena. I pesi vengono quantizzati in formato NF4 (*Nor-*

malFloat4), ottimizzato per le distribuzioni tipiche dei pesi neurali; durante il forward pass i pesi vengono de-quantizzati dinamicamente in BF16, mentre gli adattatori operano in BF16.

La Tabella 1.6 riassume i principali trade-off tra le due varianti.

Tabella 1.6: Confronto tra LoRA e QLoRA per un modello base da 7B parametri.

Caratteristica	**LoRA**	**QLoRA**
Precisione pesi base	FP16/BF16	NF4 (4-bit)
Precisione adattatori	FP16/BF16	BF16
Memoria GPU (modello 7B)	~14 GB	~4 GB
Overhead de-quantizzazione	assente	presente
Degradazione qualitativa stimata	trascurabile	<1% su benchmark principali

Prompt engineering e inference-time intervention

In alternativa al fine-tuning parametrico, è possibile adattare il comportamento di un LLM intervenendo sulla formulazione dell'input senza modificare i pesi. Tecniche come *few-shot prompting* [**brown2020language**] e *chain-of-thought prompting* [**wei2022chain**] guidano il modello verso risposte più strutturate, arricchendo il prompt con esempi o istruzioni di ragionamento. Nel sistema descritto in questa tesi, il prompt engineering è impiegato nella definizione del *system prompt* e nell'iniezione strutturata del contesto RAG; le strategie adottate e il loro impatto sulla qualità delle risposte sono analizzati nel Capitolo **??**.

1.4 Framework di orchestrazione: Haystack

La costruzione di sistemi RAG richiede l'integrazione di componenti eterogenei — modelli di embedding, database vettoriali, LLM, pipeline di pre-processing — attraverso interfacce coerenti e componibili. **Haystack** [**haystack2023**], sviluppato da deepset, è un framework Python open source progettato per questo scopo.

Nella versione 2.x adottata in questo progetto, ogni pipeline è un grafo diretto aciclico (DAG) di componenti con interfacce tipizzate, collegabili in modo dichiarativo. Sono disponibili integrazioni native con Qdrant (`QdrantDocumentStore`), Ollama (`OllamaDocumentEmbedder`, `OllamaTextEmbedder`, `OllamaGenerator`) e le principali operazioni di pre-processing (`DocumentCleaner`, `DocumentSplitter`). Il dettaglio implementativo della pipeline di ingestione e di retrieval è descritto nel Capitolo **??**.

1.5 Sintesi e posizionamento del lavoro

Il presente capitolo ha fornito il quadro teorico su cui si fonda il sistema presentato nei capitoli successivi. Le architetture Transformer costituiscono il substrato computazionale comune a componenti di generazione e embedding; i loro limiti in contesti a conoscenza chiusa — conoscenza statica, allucinazioni, privacy — motivano l'adozione del paradigma RAG. La scelta di operare con modelli e infrastrutture interamente on-premise (Ollama, Qdrant, Haystack) risponde ai vincoli di sovranità del dato e alla sostenibilità computazionale identificata nell'analisi dei requisiti. Le

tecniche di fine-tuning a parametri ridotti (LoRA, QLoRA) costituiscono il termine di confronto teorico per la valutazione delle strategie di adattamento al dominio nel Capitolo **??**.

Capitolo 2

Analisi dei Requisiti e Progettazione del Sistema

Questo capitolo definisce il caso d'uso principale, i requisiti funzionali e non funzionali, e la proposta architetturale del sistema progettato. La discussione parte dalla presentazione del contesto aziendale Intré e del modello organizzativo delle Gilde, prosegue con la specifica dei requisiti e la motivazione delle scelte tecnologiche adottate, e conclude con la descrizione dell'integrazione nel flusso di lavoro i3Guild.

2.1 Il contesto aziendale: Intré S.r.l.

Intré S.r.l.[1] è una software factory italiana con sede a Bergamo, specializzata nello sviluppo di prodotti digitali per clienti in settori eterogenei: assicurativo, automotive, trasporti, entertainment e food, tra gli altri [**intre˙website**]. Il modello operativo di Intré è fondato sul principio *Learn / Code / Deploy Value*: i team cross-funzionali operano in modalità agile e incrementale, rilasciando valore al cliente a ogni iterazione, mentre l'apprendimento continuo è strutturato come pratica organizzativa esplicita attraverso tre iniziative principali: *Camp* (eventi di formazione periodici), *Gilde* (gruppi di apprendimento interni) e *Community* (comunità di pratica aperte). L'azienda ha conseguito la certificazione ISO 27001 per la sicurezza delle informazioni e si avvale di un ecosistema di realtà correlate nelle aree cloud-native, cybersecurity, user experience design e intelligenza artificiale.

2.1.1 Il modello delle Gilde

Le **Gilde** (*guilds*) rappresentano uno dei pilastri del modello di *Learning Organization* di Intré [**intre˙gilde**]. Una Gilda è un *gruppo spontaneo di quattro-otto persone* che si forma con lo scopo di apprendere e condividere conoscenze su un argomento scelto liberamente dai partecipanti. Le Gilde hanno una durata di circa quattro mesi (un *quarter*), al termine del quale i risultati vengono presentati internamente in un evento denominato *Gildonferenza*.

Le caratteristiche salienti del modello sono:

[1] `https://www.intre.it`

- **Formazione spontanea**: ogni persona può proporre o aderire a una Gilda su qualsiasi argomento, tecnico o non tecnico (es. linguaggi di programmazione, ceramica, game development, intelligenza artificiale).
- **Ciclo temporale (Epoch)**: le Gilde si ricreano ogni quattro mesi; ogni ciclo costituisce un'"epoca" distinta, con nuovi temi e nuovi gruppi.
- **Output condivisibile**: al termine del quarter la Gilda produce un risultato tangibile (prototipo, report, repository, presentazione) che viene archiviato e condiviso.
- **Ampiezza tematica**: dall'archivio pubblico del sito aziendale emergono Gilde su temi come *Angular*, *F#*, *IA agentiva*, *game development* con Phaser, ceramica, selezione fotografica con AI, e molti altri [**intre˙gilde**].

Intré gestisce il ciclo di vita delle Gilde tramite la piattaforma web **i3Guild**, applicazione interna sviluppata con Next.js, PostgreSQL e Keycloak, che costituisce il contesto tecnologico nel quale il sistema RAG progettato in questa tesi viene integrato.

2.2 Caso d'uso: Proposta di nuove Gilde Intré

L'obiettivo è supportare utenti tecnici e non tecnici nel reperimento di informazioni aggiornate e verificabili, consentendo la generazione di risposte contestualizzate ancorate a fonti interne utili alla creazione di nuove Gilde. Il sistema adotta il paradigma *Retrieval-Augmented Generation* (RAG), introdotto da Lewis et al. [**lewis2020rag**] come metodo per estendere la memoria non-parametrica dei Large Language Model (LLM) tramite recupero dinamico di documenti da un corpus esterno.

2.2.1 Motivazione del caso d'uso

Il corpus di Gilde archiviate in i3Guild e i relativi blog post costituiscono una base di conoscenza aziendale di significativo valore: contiene descrizioni di progetti, obiettivi, rischi, risultati raggiunti e annotazioni dei partecipanti, accumulata nel corso di oltre trenta epoch a partire dal 2016 [**intre˙gilde**]. Questa conoscenza è tuttavia distribuita in un database relazionale non direttamente interrogabile in linguaggio naturale oppure in diversi blog post che non sono facilmente consultabili. Un assistente RAG permette di rendere tale patrimonio accessibile tramite query conversazionali, riducendo il tempo di ricerca e aumentando la trasferibilità della conoscenza tra i team.

2.2.2 Obiettivi del caso d'uso

- Fornire risposte affidabili e tracciabili su contenuti presenti nei repository aziendali, riducendo il rischio di allucinazioni tipico della generazione puramente parametrica [**lewis2020rag**].
- Ridurre il tempo di ricerca delle informazioni per attività operative e decisionali interne a Intré.
- Consentire l'esecuzione del servizio in ambiente on-premise per requisiti di sovranità del dato, in conformità al Regolamento Generale sulla Protezione dei Dati (GDPR) [**gdpr2016**].
- Integrare il sistema nel flusso i3Guild esistente, minimizzando l'impatto sul processo operativo consolidato.

2.2.3 Descrizione dei dati e analisi del corpus

I dati di ingresso al sistema RAG comprendono le seguenti categorie:

- **Documenti delle Gilde**: specifiche di progetto, obiettivi, rischi, risultati attesi e raggiunti, annotati con metadati strutturati (epoche, partecipanti, modalità di partecipazione).
- **Documenti tecnici**: manuali, whitepaper e specifiche di processo.
- **Glossari e vocabolari controllati**: terminologia aziendale specifica del dominio.
- **Documenti non strutturati**: file PDF, HTML e note operative con metadati associati.

Dall'analisi del corpus i3Guild (estrazione al 15/02/2026) emergono le statistiche riportate nelle Tabelle 2.1 e 2.2, rilevanti per la progettazione della pipeline di ingestione.

Tabella 2.1: Statistiche delle entità principali del corpus i3Guild (estrazione 15/02/2026).

Entità	**Conteggio**	**Note**
Guild	270	Gestibile per re-indicizzazioni complete
Epoch	30	≈ 9 guild per epoca in media
Relazioni Utente-Guild	1.166	≈ 4,3 partecipanti medi per guild
Utenti esterni	3	Trascurabile ai fini del RAG

Tabella 2.2: Statistiche dei campi testuali delle entità Guild (valori in caratteri).

Campo	**Media**	**Min**	**Max**	**Vuoti**	**Note**
`summary`	610	0	18.224	5%	Campo più ricco; outlier rilevante
`goals`	258	0	2.698	35%	Spesso breve o assente
`outcome`	133	0	2.145	36%	Spesso breve o assente
`risks`	106	0	1.245	38%	Spesso breve o assente
`achievedResultsDescription`	782	0	6.421	74%	Lungo quando presente
`journal`	≈1	1	1	99%	Praticamente inutilizzato; escluso

La somma media dei campi testuali rilevanti (`summary` + `goals` + `outcome` + `risks` + `achievedResultsDescription`) ammonta a circa 1.900 caratteri, corrispondenti a circa 400 token. Anche nel caso peggiore la somma non supera i 7.000–8.000 token, rimanendo compatibile con la finestra di contesto del modello di embedding selezionato (Sezione 2.4.3). Queste statistiche guidano direttamente la strategia di chunking e la selezione del modello.

2.3 Requisiti di sistema

I requisiti sono stati elicitati attraverso l'analisi del corpus, dei vincoli infrastrutturali di Intré e delle policy aziendali in materia di protezione dei dati. La Tabella 2.3 sintetizza i requisiti ad alto livello, suddivisi per categoria.

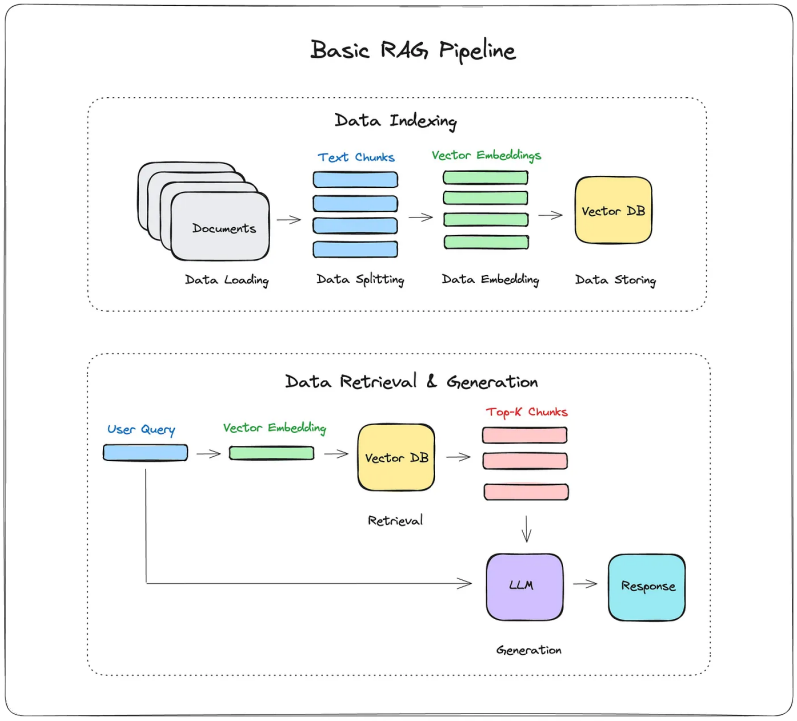

Figura 2.1: Diagramma del caso d'uso "Intré": attori, interazioni principali e confini del sistema (segnaposto).

2.3.1 Vincoli architetturali

I vincoli determinanti per le scelte architetturali sono:

- **On-premise obbligatorio**: i dati delle Gilde, incluse informazioni sui partecipanti e sui progetti, non possono transitare su infrastrutture cloud di terze parti. Il deployment locale consente di soddisfare il principio di integrità e riservatezza previsto dal GDPR [**gdpr2016**].
- **Compatibilità con lo stack i3Guild**: l'integrazione deve essere coerente con l'infrastruttura esistente (Docker Compose, PostgreSQL/Prisma, Next.js, Keycloak per l'autenticazione).
- **Limitazioni hardware**: l'ambiente Docker dispone di 8 GB di RAM complessivi, condivisi tra tutti i container (PostgreSQL, Qdrant, Ollama, Next.js, RAG service). Questo vincolo è determinante per la selezione del modello LLM (Sezione 2.4.4).
- **No API esterne per LLM ed embedding**: tutti i modelli devono essere eseguiti localmente tramite Ollama, eliminando dipendenze da provider come OpenAI o Anthropic.

Tabella 2.3: Riepilogo requisiti ad alto livello.

Categoria	**Requisiti principali**
Funzionali	Ricerca semantica full-text su documenti interni; generazione di risposte contestualizzate con riferimenti alle fonti; capacità di filtraggio per metadati (epoca, modalità di partecipazione, proprietario della guild).
Privacy e conformità	Esecuzione on-premise; nessun invio di dati a servizi cloud esterni; logging e auditing degli accessi; protezione dei dati in conformità al GDPR [**gdpr2016**]. I rischi relativi alla privacy nei sistemi RAG, tra cui la possibile estrazione dei dati di retrieval, sono discussi in letteratura [**zeng2024privacy**]; la scelta on-premise costituisce la principale misura mitigativa adottata.
Performance	Latenza di risposta nell'ordine del secondo per query semplici; throughput scalabile per carichi concorrenti moderati; inferenza CPU-only come scenario primario.
Costi e sostenibilità	Minimizzazione dei costi operativi tramite uso efficiente delle risorse (quantizzazione dei modelli, inferenza CPU); adozione di soluzioni open-source senza costi per licenze.
Manutenibilità	Componenti modulari con interfacce standardizzate; automazione della pipeline di ingestione; documentazione e test automatizzati.

2.4 Scelte tecnologiche: analisi comparativa

In questa sezione si motivano le scelte dei componenti principali dello stack tecnologico, presentando per ciascuno un'analisi comparativa delle alternative valutate.

2.4.1 Framework di orchestrazione RAG

Tre framework open-source sono stati considerati per l'orchestrazione delle pipeline RAG: **LangChain**, **LlamaIndex** e **Haystack 2.x**.

LangChain è il framework con il più ampio ecosistema di integrazioni (oltre 50.000 connettori). La sua filosofia è imperativa con componenti dichiarativi (operatore `|`), adatta per workflow agentici e multi-step reasoning. Il principale svantaggio è la curva di apprendimento elevata e la tendenza a breaking changes frequenti nelle versioni minori [**tonic2024rag**].

LlamaIndex è ottimizzato per la fase di ingestione e retrieval avanzato, offrendo oltre 150 data connector e strutture di indicizzazione specializzate. È la scelta preferibile per pipeline di question-answering puro [**athenic2025framework**].

Haystack 2.x (deepset) adotta un modello a componenti tipizzati con input/output espliciti, organizzati in pipeline dichiarative serializzabili in YAML. Studi comparativi evidenziano che Haystack offre il minore overhead di orchestrazione ($\approx$ 5.9 ms) e il minore consumo di token rispetto a LangChain ($\approx$ 10 ms) su benchmark standardizzati [**aimultiple2026benchmark**]. La

sua architettura pipeline-first favorisce la tracciabilità, i test e la sostituzione dei componenti in produzione [**peliqan2025haystack**].

Tabella 2.4: Confronto sintetico dei framework RAG candidati.

Criterio	**LangChain**	**LlamaIndex**	**Haystack 2.x**
Filosofia	Agente/imperativa	Data-first	Pipeline dichiarativa
Overhead orchestrazione	≈ 10 ms	≈ 6 ms	≈ 5.9 ms
Integrazione Qdrant	Nativa	Nativa	Nativa
Integrazione Ollama	Sì	Sì	Sì
Testabilità pipeline	Media	Media	Alta
Serializzazione YAML	No	Parziale	Sì
Adatto per produzione RAG	Sì (con overhead)	Sì	Sì (preferito)

Scelta motivata: Haystack 2.x. L'architettura a componenti tipizzati, la documentazione rigorosa, le integrazioni native con Qdrant (`QdrantDocumentStore`) e Ollama (`OllamaDocumentEmbedder`, `OllamaTextEmbedder`), il minore overhead di orchestrazione e la serializzabilità delle pipeline la rendono la scelta più adeguata per un sistema RAG production-oriented in ambiente on-premise [**tonic2024rag**].

2.4.2 Database vettoriale

La ricerca di similarità approssimata (*Approximate Nearest Neighbor*, ANN) su vettori ad alta dimensionalità è il componente infrastrutturale critico di ogni sistema RAG. Sono stati valutati quattro database vettoriali self-hostable.

Tabella 2.5: Analisi comparativa dei database vettoriali candidati.

Database	**Impl.**	**Indice**	**Filtering**	**Dashboard**	**Deployment**
Qdrant	Rust	HNSW	Pre-filtering payload	Web UI integrata	1 container
pgvector	C	IVFFlat / HNSW	SQL `WHERE`	No (via PgAdmin)	Estensione PG
Milvus	Go/C++	HNSW, IVF	Attribute-based	Attu (separato)	Multi-container
Chroma	Python	HNSW (hnswlib)	Metadata base	No	1 container

I principali criteri di esclusione e selezione sono i seguenti.

- **pgvector** è stato considerato in quanto l'infrastruttura i3Guild utilizza già PostgreSQL. Tuttavia, la letteratura segnala che le prestazioni di pgvector degradano significativamente oltre le centinaia di migliaia di vettori, con ritardi fino a 15 volte superiori rispetto a Qdrant in

termini di throughput su dataset grandi [**nirantk2024pgvector**]. Benchmark più recenti mostrano un divario più contenuto (Qdrant: ≈ 41 QPS a 99% recall su 50M vettori; pgvectorscale: ≈ 471 QPS), ma evidenziano che Qdrant ottiene latenze tail significativamente migliori (39% migliore a p95, 48% migliore a p99) [**tigerdata2025benchmark**]. Assenza di dashboard integrata e di payload filtering flessibile completano le ragioni per la non-selezione.

- **Milvus** è stato escluso per l'elevata complessità di deploy: richiede componenti aggiuntivi (`etcd`, `MinIO`), aumentando il carico operativo incompatibile con i vincoli del progetto.
- **Chroma** è implementato in Python e soffre di instabilità della persistenza dati nelle versioni precedenti; è più adatto a prototipi che a sistemi di produzione.
- **Qdrant** è scritto in Rust, garantisce prestazioni elevate e latenze stabili. La funzionalità di pre-filtering, che applica i filtri sui payload *prima* della ricerca vettoriale, è cruciale per query contestuali come "guild dell'epoca X": il filtering riduce lo spazio di ricerca prima del calcolo ANN, mantenendo il top-K accurato. Offre Web UI integrata su porta 6333, client TypeScript ufficiale compatibile con Next.js, e deploy in singolo container Docker [**qdrant2024bench**].

Scelta motivata: Qdrant. Per le ragioni sopra esposte e in linea con il vincolo aziendale di self-hosting, Qdrant rappresenta il database vettoriale più adeguato al contesto del progetto.

2.4.3 Modello di embedding

Il modello di embedding converte i documenti testuali in vettori ad alta dimensionalità che ne codificano il significato semantico. Poiché il vincolo aziendale impone l'assenza di dipendenze da API esterne, tutti i modelli candidati devono essere serviti localmente tramite Ollama.

Tabella 2.6: Confronto dei modelli di embedding candidati per il sistema RAG.

Modello	Dim.	Peso	Italiano	Contesto (tok.)	Disponibilità Ollama
nomic-embed-text	768	275 MB	Buono	8.192	Nativa
mxbai-embed-large	1.024	670 MB	Buono	512	Nativa
all-minilm	384	45 MB	Scarso	512	Nativa
multilingual-e5-large	1.024	1,3 GB	Ottimo	512	Import manuale

La selezione del modello è guidata da cinque criteri pesati: (1) qualità per la lingua italiana (peso 30%), in quanto i documenti delle Guild sono interamente in italiano; (2) footprint di risorse (25%), essenziale in un ambiente Docker con risorse condivise; (3) compatibilità nativa con Ollama (20%); (4) dimensione dello spazio vettoriale (15%); (5) benchmark MTEB [**muennighoff2023mteb**] (10%).

Scelta motivata: `nomic-embed-text`**.** Il modello, introdotto da Nussbaum et al. [**nussbaum2024nomic**], è il primo modello open-source completamente riproducibile con finestra di contesto lunga (8.192 token) che supera `text-embedding-ada-002` di OpenAI sia sul benchmark MTEB che sul benchmark LoCo per contesto lungo. Con soli 137M parametri e un peso di 275 MB, offre il miglior rapporto qualità/footprint tra i candidati. La finestra di 8.192 token è determinante: permette di ingerire la quasi totalità dei documenti consolidati delle Guild come singolo vettore senza chunking aggressivo,

preservando il contesto semantico integrale. È rilasciato sotto licenza Apache 2.0, compatibile con l'utilizzo aziendale.

2.4.4 Server di inferenza LLM e selezione del modello di chat

Il componente di generazione del testo è gestito da **Ollama**, server di inferenza locale che supporta il formato GGUF (quantizzazione a 4-bit e 8-bit). Per la selezione del modello di chat sono stati condotti test sperimentali su dodici modelli disponibili nel registry Ollama, con l'obiettivo di identificare il miglior compromesso tra qualità delle risposte e consumo di risorse nell'ambiente Docker vincolato a 8 GB di RAM totali.

Metodologia di valutazione

Il benchmark ha coinvolto dodici modelli su due categorie di query: *General* (comprensione del linguaggio, ragionamento, estrazione di entità, scrittura) e *RAG* (risposte basate su contesto recuperato). Per ogni query sono stati misurati: TTFT (*Time to First Token*), latenza totale, throughput in parole al secondo, picco di RAM e utilizzo medio della CPU. La qualità delle risposte è stata valutata su tre assi: accuratezza/ fedeltà ($Score_1$), precisione/pertinenza ($Score_2$) e coerenza ($Score_3$, valutata solo per query *General*).

Il punteggio globale (G) è calcolato come combinazione lineare pesata di quattro componenti normalizzate nell'intervallo $[0, 1]$:

$$G = 0.50 \cdot Q + 0.20 \cdot T + 0.15 \cdot (1 - R) + 0.15 \cdot (1 - C) \tag{2.1}$$

dove Q è la qualità media normalizzata, T è il throughput normalizzato, R è il consumo RAM normalizzato e C è il carico CPU normalizzato (con inversione per RAM e CPU perché valori più bassi sono preferibili).

Risultati del benchmark

La Tabella 2.7 riporta i risultati aggregati per modello, ordinati per punteggio globale G.

Motivazione della scelta: `gemma3:1b`

Alla luce dei risultati del benchmark e del vincolo di 8 GB di RAM disponibili per l'intero stack Docker, il modello selezionato è `gemma3:1b`. Le motivazioni principali sono:

- **RAM compatibile con i vincoli**: il consumo medio di circa 43 MB di RAM è contenuto, lasciando margine adeguato per gli altri container (PostgreSQL, Qdrant, Next.js, RAG service).
- **Latenza più bassa tra i modelli con qualità elevata**: latenza media di 6.74 s, la più bassa tra i modelli con un punteggio di qualità ≥ 7.5.
- **Qualità adeguata**: punteggio medio di qualità pari a 7.74, tra i più alti dell'intero benchmark.
- **Throughput elevato**: 28.01 parole/s, secondo valore più alto dopo `qwen2.5:1.5b` e `deepscaler`.

Tabella 2.7: Risultati del benchmark LLM su 12 modelli Ollama (valori medi su tutte le query). La colonna G riporta il punteggio globale calcolato secondo l'Equazione (2.1).

Modello	**Latenza (s)**	**Throughput (w/s)**	**RAM (MB)**	**CPU (%)**	**Qualità**	**G**
deepscaler:latest	18.85	29.25	42.69	1.79	7.14	83.1
llama3.2:latest	8.94	21.43	37.26	1.78	7.50	76.7
qwen2.5:1.5b	35.04	30.67	42.98	3.41	6.45	76.7
gemma3:1b	**6.74**	**28.01**	**42.83**	7.05	7.74	69.7
llama3.2:3b	16.54	20.14	40.74	2.00	6.43	68.7
gemma3:latest	17.29	16.27	42.15	3.18	7.69	66.8
granite3.3:2b	10.35	20.58	41.82	5.11	7.76	66.8
phi4-mini:3.8b	8.78	19.90	82.20	1.91	7.31	66.3
lfm2.5-thinking:1.2b	14.16	13.55	43.52	2.41	7.59	65.1
ministral-3:3b	66.79	15.09	40.26	3.12	7.19	63.4
deepseek-r1:1.5b	19.26	14.55	42.66	2.26	6.60	61.9
phi4-mini-reasoning:latest	95.53	19.55	129.14	1.87	5.00	46.8

I modelli con punteggio globale superiore (`deepscaler`, `llama3.2:latest`) presentano rispettivamente latenze elevate (18.85 s) o qualità inferiore su task RAG. Il modello `phi4-mini:3.8b`, pur mostrando buona qualità, richiede circa il doppio della RAM rispetto agli altri (82 MB), riducendo il margine disponibile per l'intero stack. La scelta di `gemma3:1b` è da considerarsi adeguata per il prototipo attuale; in un ambiente con risorse RAM superiori, modelli più capaci (es. `granite3.3:2b` o `gemma3:latest`) potrebbero essere valutati come alternativa.

2.5 Proposta architetturale

La soluzione proposta si articola in componenti modulari containerizzati: ingestione e normalizzazione dei dati, generazione di embedding, database vettoriale, orchestrazione RAG e servizio di inferenza LLM. La Figura 2.2 riporta lo schema di alto livello dell'architettura.

2.5.1 Componenti principali

Ingestione e pre-processing (`rag/ingest.py`) Pipeline batch Python che esegue le fasi ETL descritte nella Sezione 2.5.3. Acquisisce i documenti da PostgreSQL, normalizza il testo, esegue il chunking, genera gli embedding tramite Ollama e indicizza su Qdrant.

Strategia di chunking Sulla base dell'analisi dei dati (Sezione 2.2.3), viene adottato l'approccio a *documento consolidato* (1 Guild ↔ 1 documento vettoriale): tutti i campi testuali rilevanti sono concatenati in un unico documento con separatori strutturati. Per i rari outlier che eccedono 8.000 token viene applicata una *soft truncation*. Questo approccio preserva il contesto semantico integrale e semplifica la gestione di upsert e cancellazioni su Qdrant.

Database vettoriale (Qdrant) Collection principale `i3guild_knowledge` con vettori a 768 dimensioni, metrica coseno e indice HNSW. Ogni punto porta un payload JSON con metadati indicizzati per il filtering (Sezione 2.5.2).

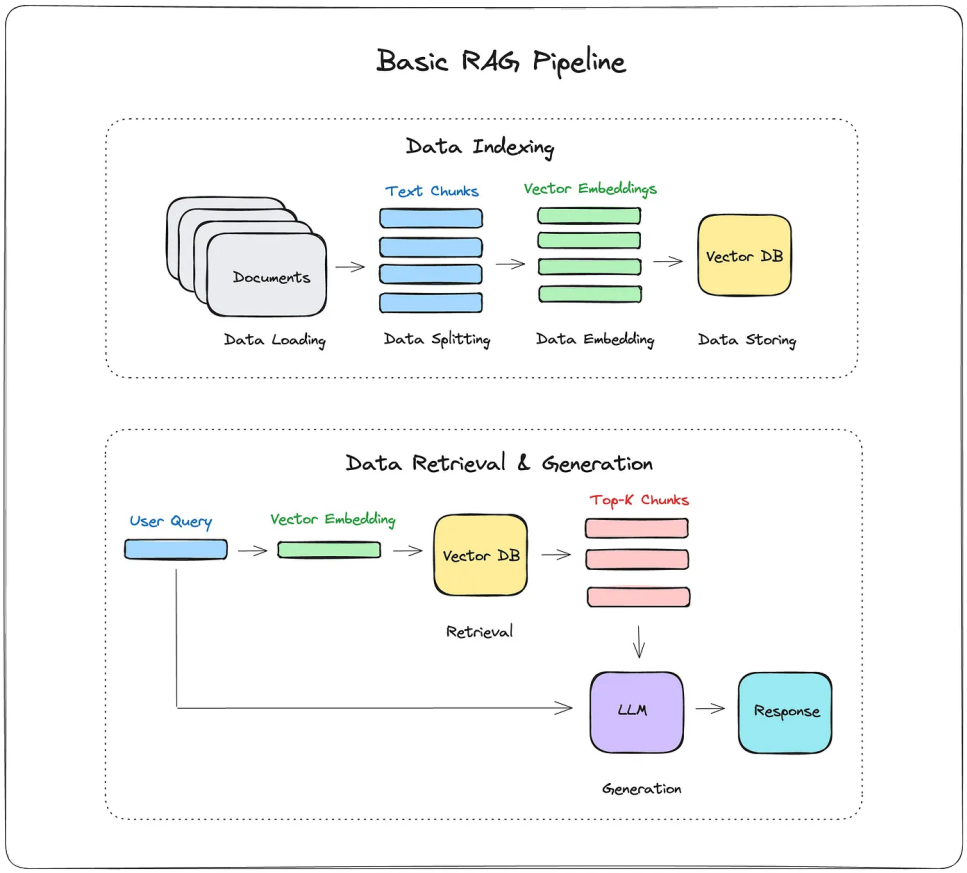

Figura 2.2: Schema architetturale del sistema RAG on-premise per i3Guild. Il flusso di ingestione (ETL) popola Qdrant a partire da PostgreSQL; il flusso di retrieval serve le query di Next.js tramite FastAPI (segnaposto).

Orchestrazione RAG (Haystack 2.x) Il container `i3guild_rag` espone pipeline Haystack per ingestione e retrieval. I componenti tipizzati (`DocumentCleaner`, `DocumentSplitter`, `OllamaDocumentEmbedder`, `QdrantDocumentStore`) sono collegati in pipeline dichiarative.

Inference LLM (Ollama) Server di inferenza locale; modello di chat adottato: `gemma3:1b`, selezionato sulla base del benchmark sperimentale condotto su dodici modelli (Sezione 2.4.4).

Servizio API (FastAPI) Microservizio Python che espone l'endpoint `POST /rag/retrieve` (porta 8000, mappata a 8002 su host). Restituisce documenti con score di similarità e campi diagnostici.

Osservabilità e sicurezza Logging centralizzato, metriche di latenza e utilizzo risorse; autenticazione tramite Keycloak (già presente in i3Guild); cifratura a riposo per i dataset sensibili.

2.5.2 Progettazione della collection Qdrant

La collection `i3guild_knowledge` è configurata con i parametri riportati nella Tabella 2.8.

Tabella 2.8: Parametri di configurazione della collection Qdrant.

Parametro	Valore	Motivazione
`vectors.size`	768	Output di `nomic-embed-text`
`vectors.distance`	`Cosine`	Standard per similarità semantica testuale
`hnsw_config.m`	16 (default)	Compromesso velocità/precisione
`hnsw_config.ef_construct`	100 (default)	Qualità dell'indice in fase di build

Ogni punto nella collection porta un payload JSON con i campi elencati nella Tabella 2.9. I campi marcati come indicizzati abilitano il pre-filtering, caratteristica distintiva di Qdrant che applica i filtri sul payload *prima* della ricerca vettoriale, garantendo che il calcolo del top-K sia effettuato solo sul sottoinsieme filtrato.

Tabella 2.9: Struttura del payload Qdrant per la collection `i3guild_knowledge`.

Campo	Tipo	Descrizione	Indicizzato
`guild_id`	integer	ID della guild sorgente	Sì
`guild_name`	keyword	Nome della guild	Sì
`epoch_id`	integer	ID dell'epoca di appartenenza	Sì
`epoch_start`	datetime	Data inizio epoca	Sì
`epoch_end`	datetime	Data fine epoca (nullable)	Sì
`participation_mode`	keyword	FullRemote / Hybrid / OnSite	Sì
`is_individual`	bool	Guild individuale	Sì
`owner_id`	keyword	ID del proprietario	Sì
`participants`	keyword[]	Nomi dei partecipanti	Sì
`participant_count`	integer	Numero di partecipanti	No
`source_text`	text	Testo originale del chunk	No
`last_synced`	datetime	Timestamp ultima sincronizzazione	No

Gli ID dei punti sono **UUID deterministici** generati tramite hash di `guild_id` + `field_source` + `chunk_index`, garantendo l'idempotenza degli upsert: lo stesso documento produce sempre lo stesso ID, prevenendo la creazione di duplicati in caso di re-indicizzazione.

2.5.3 Pipeline di ingestione ETL

Lo script `rag/ingest.py` esegue le seguenti fasi:

1. **Extract**: connessione a PostgreSQL e query SQL con JOIN su `Epoch`, `GuildUserRelation` ed `ExternalUser` per estrarre tutti i campi semantici delle Guild insieme ai nomi dei partecipanti.

2. **Transform**: applicazione di `DocumentCleaner` (stripping HTML dai campi WYSIWYG come `journal`), normalizzazione degli spazi, rimozione di segnaposti residui; `DocumentSplitter` con chunking a 150 parole quando necessario (outlier), con overlap del 10–20%.

3. **Embed**: generazione degli embedding tramite `OllamaDocumentEmbedder` (modello `nomic-embed-text`, endpoint interno Docker `http://ollama:11434`).
4. **Load**: scrittura su Qdrant tramite `DocumentWriter` con policy `DuplicatePolicy.OVERWRITE`.

Il seguente frammento illustra la struttura della pipeline Haystack:

Listing 2.1: Struttura della pipeline di ingestione Haystack 2.x (pseudocodice).

```
from haystack import Pipeline
from haystack.components.preprocessors import DocumentCleaner, DocumentSplitter
from haystack.components.writers import DocumentWriter
from haystack_integrations.components.embedders.ollama import OllamaDocumentEmbedder
from haystack_integrations.document_stores.qdrant import QdrantDocumentStore

qdrant_store = QdrantDocumentStore(url="http://qdrant:6333",
                                   collection_name="i3guild_knowledge",
                                   embedding_dim=768)

indexing_pipeline = Pipeline()
indexing_pipeline.add_component("cleaner", DocumentCleaner())
indexing_pipeline.add_component("splitter", DocumentSplitter(
    split_by="word", split_length=500, split_overlap=50))
indexing_pipeline.add_component("embedder", OllamaDocumentEmbedder(
    model="nomic-embed-text",
    url="http://ollama:11434/api/embed",
    generation_kwargs={"num_ctx": 8192}))
indexing_pipeline.add_component("writer", DocumentWriter(
    document_store=qdrant_store,
    policy=DuplicatePolicy.OVERWRITE))

indexing_pipeline.connect("cleaner.documents", "splitter.documents")
indexing_pipeline.connect("splitter.documents", "embedder.documents")
indexing_pipeline.connect("embedder.documents", "writer.documents")

indexing_pipeline.run({"cleaner": {"documents": raw_docs}})
```

Il comando per avviare l'ingestione da host (stack Docker attivo) è:

```
docker-compose exec i3guild_rag python rag/ingest.py
```

2.5.4 Pipeline di retrieval e logica di ranking

La pipeline di retrieval (`rag/api.py`) implementa una ricerca ANN con logica adattiva per la selezione del numero di documenti restituiti. Il sistema non adotta un valore `top_k` fisso; applica invece un approccio a tre stadi:

1. **Soglia di similarità** (default: cosine score ≥ 0.5): esclude documenti semanticamente non pertinenti.

2. **Rilevamento del gap**: se il salto di score tra due documenti consecutivi supera una soglia configurabile (default: 0.08), la lista viene troncata al confine naturale della rilevanza.
3. **Cap di sicurezza** (default: ≤ 10 documenti): limite massimo per controllare la dimensione del contesto passato al modello generativo.

Query rewriting e Topic-First Search

In corpora domain-specific come i3Guild, alcuni termini frequenti in quasi tutti i documenti (es. "gilda", "fondare", "consigli") dominano lo spazio degli embedding, producendo risultati generici in luogo di quelli pertinenti. Il meccanismo di *query rewriting* implementato nella funzione `extract_topic_query()` risolve questo problema attraverso le seguenti fasi:

1. Normalizzazione e rimozione della punteggiatura.
2. Filtraggio di un insieme di circa 80 *domain stopwords* (`DOMAIN_STOPWORDS`): termini specifici del dominio i3Guild e stopwords italiane generiche.
3. Estrazione del topic residuo; se diverso dalla query originale, la ricerca viene eseguita sul topic pulito (*topic-first strategy*).

La Tabella 2.10 illustra l'effetto del rewriting su query campione.

Tabella 2.10: Effetto del query rewriting su esempi di query utente.

Query originale	**Topic estratto**	**Risultato atteso**
"vorrei fondare una gilda sulla musica, cosa mi consigli?"	`musica`	Guild a tema musicale
"intelligenza artificiale"	`intelligenza artificiale`	(nessun rewriting)
"chi ha partecipato al progetto di design?"	`design`	Guild con partecipanti nel settore design

2.5.5 Endpoint API di riferimento

L'endpoint principale esposto dal microservizio FastAPI è il seguente:
POST `/rag/retrieve`
Request body (esempio):

Listing 2.2: Esempio di request body per l'endpoint `/rag/retrieve`.

```
{
  "query":            "vorrei fondare una gilda sulla musica",
  "top_k":            20,
  "score_threshold":  0.5,
  "max_documents":    10,
  "gap_threshold":    0.08,
  "filters":          null
}
```

Response body (struttura):

Listing 2.3: Struttura della response dell'endpoint `/rag/retrieve`.

```
{
  "documents":          [ { "content": "...", "score": 0.85,
                            "meta": { "guild_name": "...",
                                      "participants": [...] } } ],
  "total_candidates": 15,
  "threshold_used":   0.5,
  "cutoff_reason":    "gap",
  "topic_query":      "musica"
}
```

I campi `total_candidates`, `threshold_used`, `cutoff_reason` e `topic_query` (presente solo quando il rewriting è attivo) sono utili per il debug e la valutazione sperimentale del sistema (Capitolo **??**).

2.6 Integrazione con i3Guild

L'integrazione del sistema RAG nell'infrastruttura i3Guild esistente avviene senza modifiche strutturali ai componenti già operativi (PostgreSQL, Next.js, Keycloak, Kong). Il sistema RAG si aggiunge come sottoinsieme di servizi nel file `dev_containers/docker-compose.yml` esistente.

2.6.1 Servizi Docker aggiuntivi

La Tabella 2.11 elenca i container aggiuntivi introdotti dal sistema RAG.

Tabella 2.11: Servizi Docker introdotti dall'integrazione RAG in i3Guild.

Container	**Immagine**	**Porta host**	**Ruolo**
`qdrant`	`qdrant/qdrant:latest`	6333 (REST), 6334 (gRPC)	Database vettoriale; Web UI su `/dashboard`
`ollama`	`ollama/ollama:latest`	11434	Server di inferenza LLM ed embedding
`i3guild_rag`	Build locale	8002	FastAPI + Haystack (retrieval e ingestione)
`postgres`	già presente	già presente	Source of truth relazionale (nessuna modifica)

2.6.2 Variabili d'ambiente

2.6.3 Integrazione lato Next.js

Lato applicazione Next.js, il sistema RAG è integrato tramite il servizio TypeScript `src/lib/services/rag.service.ts`, che espone la funzione `searchRAG(query, filters?, options?)`. Questa funzione chiama l'end-

Tabella 2.12: Variabili d'ambiente introdotte dalla componente RAG.

Variabile	**Valore di default**	**Descrizione**
`QDRANT_URL`	`http://localhost:6333`	Endpoint REST Qdrant
`QDRANT_COLLECTION_NAME`	`i3guild_knowledge`	Collection principale
`OLLAMA_BASE_URL`	`http://localhost:11434`	Endpoint Ollama
`OLLAMA_EMBEDDING_MODEL`	`nomic-embed-text`	Modello di embedding
`OLLAMA_CHAT_MODEL`	`gemma3:1b`	Modello LLM di chat (selezionato da benchmark)
`EMBEDDING_DIMENSIONS`	`768`	Dimensione vettore
`RAG_API_URL`	`http://i3guild_rag:8000`	Endpoint interno del servizio RAG

point Python `/rag/retrieve` con i parametri di retrieval dinamico e restituisce un array vuoto in caso di irraggiungibilità del servizio Python, garantendo la *graceful degradation* del chatbot.

Le chiamate a Ollama per la generazione del testo utilizzano un agent `undici` personalizzato con `headersTimeout: 300s` per evitare il `HeadersTimeoutError` che si manifesta al primo avvio, quando Ollama carica il modello in memoria (latenza tipica di 60–120 s su CPU-only).

2.6.4 Sincronizzazione dei dati

PostgreSQL è la *source of truth*; Qdrant è una vista derivata che deve essere mantenuta sincronizzata. Sono state identificate tre strategie di sincronizzazione, descritte in ordine crescente di complessità implementativa. La strategia raccomandata è il **full re-index periodico**: lo script `rag/ingest.py` riscrive l'intera collection su base giornaliera o settimanale. È semplice da implementare, privo di rischi di inconsistenza e accettabile grazie alla natura locale dell'embedding (nessun costo per token). L'evoluzione verso un *sync incrementale* (solo guild modificate, identificate tramite `@updatedAt` di Prisma) o uno *sync event-driven* (trigger su ogni operazione CRUD) è pianificata come sviluppo futuro.

Capitolo 3

Implementazione del Sistema RAG

* Pipeline di Ingestion: Pre-processing dei documenti, chunking strategies e generazione degli embedding. * Integrazione LLM: Configurazione dell'infrastruttura locale (on-premise) e gestione dei System Prompt. * Sviluppo dell'Agente Cognitivo: Logica di gestione del contesto e conversazione multi-turno. * Gestione fonti eterogenee: Integrazione di dati strutturati e non strutturati (esiti delle Gilde).

Capitolo 4

Metodologia di Ricerca e Ottimizzazione

* Ottimizzazione dell'Attention: Strategie per il trattamento del glossario tecnico aziendale. * Inference-Time Intervention: Sperimentazione sulla manipolazione dei vettori di attivazione (come discusso con la Professoressa). * Strategie di Quantizzazione: Impatto della quantizzazione (4-bit vs 8-bit) sulla qualità della generazione in contesti specialistici. * Fine-tuning (Opzionale): Analisi di fattibilità con risorse limitate (PEFT/

Capitolo 5

Analisi Sperimentale e Valutazione dei Risultati

* Dataset di Validazione: Definizione di una "Ground Truth" per le domande sulle Gilde. * Metriche Quantitative: * Retrieval: Hit Rate, MRR (Mean Reciprocal Rank). * Generation: Faithfulness, Answer Relevance (tramite framework come RAGAS). * Analisi Qualitativa: Studio dei casi di fallimento (allucinazioni, errori di retrieval) e feedback degli stakeholder di Intré. * Benchmarking: Confronto tra LLM diversi (es. OpenAI vs Llama locale).

Conclusioni e Sviluppi Futuri

* Sintesi del lavoro: Efficacia del sistema RAG nell'ecosistema aziendale. * Contributi originali: Risultati dell'analisi metodologica sull'attenzione e l'inferenza. * Limiti e lavori futuri: Agenti multi-modali, miglioramento continuo della base di conoscenza.

Appendice A

Annotazioni e Convenzioni

Text

Appendice B

Codice Utilizzato

Text

Bibliografia